

Group Project Report – PART A Python Unit Testing

**Andrew Truong, Joey Gonzalez, Hubert Tran, Toan Tran, Vance Tran,
Gerrit Vanderstoel, Courtney Vu**

CPSC 463-01 - Software Testing

Professor Ning Chen

Department of Computer Science

California State University, Fullerton

Fall 2025

Table of Contents

1.0 Introduction	3
2.0 Initial Struggles	4
3.0 Unit Testing the Module	6
4.0 Conclusion	8
References	9

1.0 Introduction: World of Unit Testing

Our group project consists of us getting our hands dirty with Odoo and the world of unit testing. The main goal is to learn about how unit tests work and their importance in the real world. What exactly is Odoo and how is it used? Odoo and specifically Odoo 15 is a full-stack business management platform that bundles ERP, CRM, accounting, inventory, HR, website, and more into a single system. It's built on Python and PostgreSQL, with a modular structure; each feature is a module you can enable or extend. The framework includes its own ORM, web client, API layers, automated workflows, and templating engines. Version 15 specifically brought performance improvements, faster UI, updated website builder, better POS, and a more polished backend compared to earlier versions. Developers use it as a base to build custom business logic, automation, and integrations.

What is unit testing exactly? Unit testing is the process of checking small, isolated pieces of code, usually functions, classes, or methods, to ensure they behave as expected. A “unit” should run without relying on the database, HTTP, or external systems unless you’re specifically testing those layers. Unit tests are automated, repeatable, and run quickly, helping developers catch errors early in the development cycle. In frameworks like Odoo, unit testing ensures your business rules, computations, and helper methods work correctly before integrating with the rest of the system.

Why does unit testing matter? Testing catches logic errors before users see them; without tests, regressions slip in easily. Business systems like Odoo handle money, stock, and critical processes. Obviously, mistakes in the real-world cost real money. Automated tests make refactoring safer because you immediately know when a change breaks something. They enforce

consistent behavior across modules, especially in long-lived projects. Good test coverage reduces debugging time and increases confidence in deployments, upgrades, and new features.

2.0 Initial Struggles

Initially, our first step was installing and setting up Ubuntu to run Odoo 15. While not the most trivial step, it was still something that we had to do. While some of us have some experience with Linux, we mostly come from a Windows background. We had the options of either running Ubuntu natively on a storage device off our machines or running Ubuntu virtually using a virtual machine. Most of us opted for virtual machines as they are simpler and offer quality of life features.

Once that was up and running, we installed Ubuntu and were ready to proceed with the installation of Odoo 15. However, our biggest hurdle had to be the setup of Odoo itself. Since it was our first time installing and using Odoo, we relied on tutorials found online. However, these tutorials all varied in versions, clarity, and the steps themselves, with the outcome of some not working on our machines. Even though the professor had given us some links with tutorials to start off, we found out that they did not work on our machines and we had to find our own. Luckily, we managed to find good ones, and we eventually got them working. Many packages had to be installed for Ubuntu and Python in a virtual environment. Because of this, we encountered various errors and dependencies issues when installing Odoo, but we managed to figure them out with the help of each other, online resources, or ChatGPT.

Team members had an issue where the Python dependencies and the Python version itself were incompatible with Ubuntu version 22.04 LTS. After trial and error with some research, they

found the solution of reverting back manually to previous Python versions 3.7 - 3.9. Especially, packages software-properties-common and repository ppa:deadsnakes/ppa which had to be sourced externally. Issues with the command Python3 would result in errors, however using Python3.9 resolved these issues.

Another issue that we identified when installing Odoo was the permissions of folders and files, sometimes not allowing Odoo to run properly or find plugins, but once again we figured it out by allowing the Odoo user to use the proper files and folders that it needs.

Remember to enable and set up a log file to output any information and errors that may be important. We realized if this log file was not set and we would have no idea what was happening behind the scenes and what we needed to fix. This is called silent mode and while useful if everything is running smoothly, in this case it was another roadblock in our way. Enabling this log file can be found in the config file etc/odoo15.conf. This makes debugging much easier and outputs info to /var/log/odoo15/odoo.log. Despite the setback and errors, which were tedious, were fixed and we were left with a properly working Odoo environment.

An error that occurred for one of the team members that left him stuck on the database creation page where he was told to make a database using the configuration file information that was made during the installation of Odoo. To fix that type of error, his problem was that he needed to update his files by adding “-u” into his launch.json file so the database would update. After doing so, he has had no issues and just left it with the -u so that no other update problem would occur. Issues like this take a lot of time to resolve because it shows that something small can have an impact on how the program functions.

3.0 Unit Testing the Module

There are three types of unit tests we are able to perform; they all have different uses and advantages, each with their own place and time to use. There are transaction cases, save point cases, and http cases.

- ❖ Transaction case

- Runs each test inside a database transaction. Database changes are rolled back after each test, which results in a clean slate every time. Best used for testing model methods that create/update/delete records, check constraints, access rights, change and computed fields. The only problem is it results in lower than pure unit tests because DB setup/teardown is involved. However, it is very reliable and isolated, which means tests don't affect each other.

- ❖ Save point case

- Database save points are used to avoid recreating the whole DB state for each test. Each test starts from a save point and rolls back to it. This is best used for large datasets or heavy initialization logic. Test suites where many tests share the same base environment. Faster than pure transaction-per-test when the setup cost is high. Still gives isolation at the test level thanks to rolling back to the save point.

- ❖ Http case

- Test client / environment to call Odoo controllers. Simulated: HTTP methods: GET/POST, sometimes JSON-RPC. Different users/sessions, including public/portal/internal users. Best for: Testing routes, controllers, and website flows. Verifying authentication, permissions, and redirects. Checking

responses: status codes, templates rendered, JSON payloads. Slower and more complex than pure unit tests, so only used where HTTP behavior actually matters.

❖ How Odoo tests with Python's unit test

- Test classes inherit from Python's unit test. Test Case-based Odoo helpers (e.g. TransactionCase, SavepointCase, HTTP-specific base classes). Tests follow the standard pattern: setup class to prepare environment and data. Methods starting with test_ for actual tests. Use of assert equal, assertTrue, assertRaises, etc. Odoo provides: A test environment with self.env for ORM access. Helpers to create users, records, and simulate contexts (with context, sudo, etc.). Unlike common build-in unittest framework used in other Python scripts, Odoo unittest is unable to run like the usual Python3 `-m unittest test_sample`. Tests are run with Odoo's test runner: Via command line flags (e.g. `--test-enable`, module-specific test options). Integrated into CI to ensure modules don't break.

❖ Test folder structure

- Tests live inside each module: Typical path: `/tests/` Common patterns: **init.py** to import test files. Files named like `test_*.py` (e.g. `test_sale_order.py`, `test_access_rights.py`). Separation by concern: Model tests (`test_models.py`, `test_py`). Controller/HTTP tests (`test_controllers.py`). Security/access tests (`test_security.py`). Keep tests close to the code they validate: Easier to maintain. Easier to run module-only test suites.

4.0 Conclusion

Working through this project forced our group to confront the real challenges and setbacks behind installing and using a complex platform like Odoo. The setup process alone exposed issues with system dependencies, Python versions, permissions, and configuration error. These problems don't really show up until you get your hands dirty in an actual environment or real-world scenario. Solving these roadblocks gave us a much clearer picture of why development teams rely so heavily on proper hands-on experience and collaboration.

Once we reached the unit testing phase, the value of Odoo's testing framework became very obvious. Learning the differences between TransactionCase, SavepointCase, and HttpCase showed that not all tests serve the same purpose; each exists to check specific behaviors at the right level of isolation. Understanding how Odoo integrates Python's unittest really clarified how real-world systems can use a simple language such as Python to enforce reliability and keep large codebases maintainable over time.

Overall, this project gave us hands-on experience with both the technical and problem-solving sides of software testing. The process was tedious at times, but by the end, we gained a stronger grasp of how software testing is used and how important it is when it comes to software development.

References

2Pointers. (2023, June 15). *Exploring Python Unit Testing in Odoo*. Medium.

<https://2pointers.medium.com/exploring-python-unit-testing-in-odoo-c87b348778dd>

Odoo Mates. (2019, December 5). *Install PgAdmin Configure With Odoo Access, Visualize and*

Update Data. YouTube. <https://www.youtube.com/watch?v=TWccA-tVa9E>

Testing Odoo — Odoo 15.0 documentation. (2025). Odoo.com.

<https://www.odoo.com/documentation/15.0/developer/reference/backend/testing.html#testing-js-code>

Thạch Phạm. (2022, May 13). *How to install Odoo 15 on Ubuntu 22.04*. AZDIGI Blog.

<https://azdigi.com/blog/en/linux-server-en/web-server/how-to-install-odoo-15-on-ubuntu-22-04/>